

Generative Neural Networks

S. R. Mahadeva Prasanna

Professor, Dept. of DSAI & Director, IIIT Dharwad

prasanna@iiitdwd.ac.in

October 29, 2024

Some Good Resources for NNDL

- Mithesh M. Khapra, "Deep Learning Part I and II" NPTEL Course.
- Ali Ghodsi, "Deep Learning", University of Waterloo Course.
- Duda and Hart, "Pattern Classification" Wiley, 2001.
- C. M. Bishop, "Neural Networks for Pattern Recognition", 1995.
- B. Yegnanarayana, "Artificial Neural Networks", PHI, 1999.
- C. M. Bishop, "PRML", Springer, 2006.
- Ian Goodfellow, et. al. , "Deep Learning", MIT Press, 2016.
- C. M. Bishop & H. Bishop, "Deep Learning", Springer, 2024
- Simon J. D. Prince, "Understanding Deep Learning", MIT Press, 2024



Organization

- Introduction
- Autoencoders
- Variational Autoencoders
- Generative adversarial network (GAN)
- Generative AI using transformers
- Restricted Boltzmann machine (RBM)

Significance of Generative Neural Networks

- AI models are data hungry, need more data
- Collection of large amount of data involves time and costly
- Any amount of data that can be physically collected is still less
- Generative AI models can generate more data

Learning and Modeling

- Supervised vs unsupervised learning.
- Generative vs discriminative modeling.
- **Unsupervised**: Clustering, dimensionality reduction, feature learning.
- **Generative Models**: Learn nature of distribution from given training data and then generate new data from same distribution.
- Generative models for **Explicit vs Implicit** distribution modeling.
- Usefulness of Generative models is we can generate more samples from the same distribution
- Generative artificial intelligence (GenAI)



About Autoencoders

- Special type of FFNN having encoder-decoder architecture.
- **Encoder** encodes input $\mathbf{x}$ to hidden representation $\mathbf{h}$, $\mathbf{h} = g(W\mathbf{x} + \mathbf{b})$.
- **Decoder** hidden repr. $\mathbf{h}$ back to appr. input $\hat{\mathbf{x}} = f(W^*\mathbf{h} + \mathbf{c})$.
- **Main point is $\dim(\mathbf{h}) \ll \dim(\mathbf{x})$.**
- Lower dim repr $\mathbf{h}$ contains most infmn. of higher dim input $\mathbf{x}$.
- Train such a FFNN which will minimize error between $\mathbf{x}$ and $\hat{\mathbf{x}}$.
- $\mathbf{h}$ got all the redundant dim removed.

Nature of Loss Function

- $\min_{W, W^*, b, c} L = \frac{1}{M} \sum_{i=1}^M \sum_{j=1}^N (\hat{x}_{ij} - x_{ij})^2$, M of data points, N is dim.
- $\min_{W, W^*, b, c} L = \frac{1}{M} \sum_{i=1}^M \sum_{j=1}^N (\hat{x}_{ij} - x_{ij})^T (\hat{x}_{ij} - x_{ij})$
- Loss function L is different compared to MLP.
- Except for this, everything else remains same. Accordingly, we can use backpropagation for computing the error and weight adjustment.

Variational Autoencoders

- Can autoencoders act as generative model?
- Yes in principle, using decoder part of autoencoder.
- Instead of encoding an input as a single point, **encode it as a distribution over the latent or hyperparameter space.**
- How to train such an autoencoder ?
 - Input is encoded as distribution over the latent space.
 - A point from the latent space is sampled from that distribution.
 - Sampled point is decoded and reconstruction error can be computed.
 - Reconstruction error is backpropagated through the network.



Variational Autoencoders

- NNs interpreted from a stochastic or probabilistic point of view.
- Operation of autoencoder viewed from a sampling of distribution.
- $p(h/X)$ from encoder point of view.
- Neural network based encoder to learn the distribution $p(h/X)$.
- Similarly, $p(X/h)$ for decoder point of view.
- Neural network based decoder to learn the distribution $p(X/h)$.

Variational Autoencoders

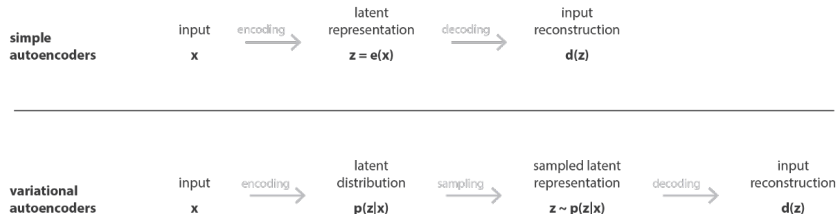


Figure: Autoencoder vs Variational Autoencoder

What Distribution to Model ?

- Encoded distribution chosen to be normal, encoder trained to return mean and covariance matrix that describe these Gaussians.
- Ensure both a local and global regularisation of latent space (local due to variance control and global due to mean control).
- **Loss function** : a “reconstruction term” (on the final layer), that tends to make the encoding-decoding scheme as performant as possible, and a “regularisation term” (on the latent layer), that tends to regularise the organisation of the latent space by making the distributions returned by the encoder close to a standard normal distribution.
- Regularisation term is expressed as the Kulback-Leibler divergence between the returned distribution and a standard Gaussian.

Continuity and Completeness

- Regularity expected from latent space in order to make generative process possible can be expressed through two main properties.
- **Continuity** : Two close points in the latent space should not give two completely different contents once decoded.
- **Completeness** : For a chosen distribution, a point sampled from the latent space should give “meaningful” content once decoded.
- The only fact that VAEs encode inputs as distributions instead of simple points is not sufficient to ensure continuity and completeness.
- Need a well defined regularization term.

Understanding VAE by Joseph Rocca.



Significance of Regularization

- Without a well defined regularisation term, the model can learn and behave almost like classic autoencoders (leading to overfitting).
- Encoder can either return distributions with tiny variances or return distributions with very different means.
- In both cases, distributions are used the wrong way and continuity and/or completeness are not satisfied.
- Regularise both covariance matrix and mean of distributions returned by encoder.
- In practice, done by enforcing distributions to be close to a standard normal distribution (centred and reduced).
- Covariance matrices to be close to identity, preventing punctual distributions, and the mean to be close to 0, preventing encoded distributions to be too far apart from each others.

Understanding VAE by Joseph Rocca.

Information, Entropy, KLD

- Information, $I = -\log p(x)$
- Entropy, $H = -\sum p(x)\log p(x)$
- KL Divergence, $KL(p||q) = -\sum q(x)\log q(x) + \sum p(x)\log p(x)$
- However, KLD is taken wrt to one distribution, say $p(x)$
- $KL(p||q) = -\sum p(x)\log q(x) + \sum p(x)\log p(x)$
- $KL(p||q) = -\sum p(x)\log\left(\frac{q(x)}{p(x)}\right)$
- $KL \geq 0$ and $KL(p||q) \neq KL(q||p)$
- $p(z/x) = \frac{p(x/z)p(z)}{p(x)}$
- Computing $p(x)$ is difficult as $p(x) = \int p(x/z)p(z)dz$
- How to deal with it ?



Variational Information

- How to deal with it ?
- Either by Gibbs sampling or by variational inference (VI)
- Gibbs sampling is unbiased estimator with large variance
- Variational inference is biased estimator with negligible variance
- VI procedure : Assume $p(z)$ from a std family of distribution
- Then play around parameters of the distribution such that $p(z)$ becomes closer to $p(z/x)$

Mathematical Framework for VAE

- Let x represents data and assume x is generated from a latent variable z .
- **Generation process :**
 - A latent representation z is sampled from the prior distribution $p(z)$
 - Data x is sampled from the conditional likelihood distribution $p(x|z)$
- **Probabilistic decoder :** defined by $p(x|z)$, describes distribution of decoded variable given the encoded one.
- **Probabilistic encoder :** defined by $p(z|x)$, describes distribution of encoded variable given the decoded one.
- Encoded representations z in the latent space are assumed to follow the prior distribution $p(z)$

Mathematical Framework for VAE

- Using **Bayes theorem**

- $p(z/x) = \frac{p(x/z)p(z)}{p(x)}$

- $p(z/x) = \frac{p(x/z)p(z)}{\int p(x/u)p(u)du}$

- Let $p(z)$ is a standard Gaussian distribution.

Understanding VAE by Joseph Rocca.

Mathematical Framework for VAE

- As we know $p(z)$ and $p(x|z)$, we can use the Bayes theorem to compute $p(z|x)$
- However, this kind of computation is often intractable (because of integral at the denominator).
- Therefore require use of approximation techniques such as **variational inference**.

Understanding VAE by Joseph Rocca.



Variational Inference Formulation

- Variational inference (VI) is a technique to approximate complex distributions.
- Set a parametrised family of distribution and to look for the best approximation of our target distribution among this family.
- Best element in the family is one that minimise a given approximation error measurement (K-L divergence between approximation and target).

Understanding VAE by Joseph Rocca.

Variational Lower Bound

- Difficulty in estimating $p(z/x)$ as computation of $p(x)$ is difficult to compute.
- Let us assume a distribution $q(z)$ and minimize the KLD with $p(z/x)$
- This is approximation of distribution.
- $\min(q(z)||p(z/x)) = -\sum q(z) \log \frac{p(z/x)}{q(z)}$.
- $\min(q(z)||p(z/x)) = -\sum q(z) \log \frac{\frac{p(z,x)}{p(x)}}{q(z)}$.
- $\min(q(z)||p(z/x)) = -\sum q(z) \log \left(\frac{p(z,x)}{q(z)} \frac{1}{p(x)} \right)$.
- $\min(q(z)||p(z/x)) = -\sum q(z) \log \left(\frac{p(z,x)}{q(z)} \right) - \sum q(z) \log \left(\frac{1}{p(x)} \right)$.



Variational Lower Bound

- $\min(q(z)||p(z/x)) = -\sum q(z)\log(\frac{p(z,x)}{q(z)}) + \sum q(z)\log(p(x)).$
- $\min(q(z)||p(z/x)) = -\sum q(z)\log(\frac{p(z,x)}{q(z)}) + \log p(x)\sum_z q(z).$
- $\min(q(z)||p(z/x)) = -\sum q(z)\log(\frac{p(z,x)}{q(z)}) + \log p(x).$
- $\log p(x) = \min(q(z)||p(z/x)) + \sum q(z)\log(\frac{p(z,x)}{q(z)}).$

Variational Lower Bound

- $\log p(x) = KLD(q(z)||p(z/x)) + \sum q(z) \log(\frac{p(z,x)}{q(z)})$.
- LHS is a const., first term on RHS is KLD and second term is variational lower bound.
- Minimize first term is equivalent to maximizing second term VLB.
- KLD always greater than 0. Therefore second term VLB should be less than or equal to $\log p(x)$.

Variational Lower Bound

- $\sum q(z) \log\left(\frac{p(z, x)}{q(z)}\right) = \sum q(z) \log\left(\frac{p(x/z)p(z)}{q(z)}\right).$
- $\sum q(z) \log p(x/z) + \sum q(z) \log \frac{p(z)}{q(z)}.$
- $E_q(z) \log p(x/z) + KLD(p(z) || q(z)).$

Variational Autoencoders

- Assume some distribution, say a Normal distribution $N(\mu, \Sigma)$.
- Parameters will be μ and Σ
- $p_{\theta}(h/X)$, where θ represent pars of encoder.
- Similarly, $p_{\phi}(X/h)$, where ϕ represent pars of decoder.
- What is the objective function for such a case?
- Maximizing probability !
- Will it be applicable to both distributions that will be captured?
- Yes, by adding additional term in objective function, first term for decoder prob maximization and second term for minimizing KLD between encoder distbn and actual dstbn.



Variational Autoencoders

- Encoder captures distribution, mean μ and variance Σ .
- Value of h can be obtained by sampling the distribution depicted by μ and Σ .
- Those values of h that give generation of new data as closely as possible to input X .
- That is maximization of probability $p(X/h)$

Generative Adversarial Network

- An implicit generative model.
- Start from some random samples distribution and learn underlying distribution of given training data.
- Can be achieved using neural network.
- Goal is input to NN is samples of some random distribution and output is samples of training data !
- Such a NN is termed as Generative network.
- Very attractive alternative to explicit generative model like stochastic model (GMM / HMM).



Maximum Likelihood Modelling

- Maximum likelihood estimation of parameters.
- Estimate the parameters θ of the model that maximizes the belongingness of random samples / features to that distribution.
- $\theta^* = \arg \max_{\theta} E(\log P(x/\theta, \lambda))$
- **Explicit modeling** : Parameters θ are for a specific model or distribution or a priori assumption about model λ , ex, GMM, HMM,
- **Implicit modeling** No a priori assumption about the specific model or distribution. Same is learnt from data.
- Model parameters are learnt implicitly by unsupervised discovery with the help of adversarial behavior from another network.



Link from Variational Autoencoder

- Given x , use encoder to learn parameters θ as result of minimizing distance between assumed normal distribution for hidden pars $p(h)$ and conditional distribution $p_{\theta}(h/x)$.
- The decoder to learn parameters ϕ as result of maximizing the probability of estimating x from the pars of h , i.e., θ .
- An alternative view is for GAN
- Starting from normal distribution, adjust the pars θ_g for constructing x using discriminating approach.

Discriminator function $D(x)$ of GAN

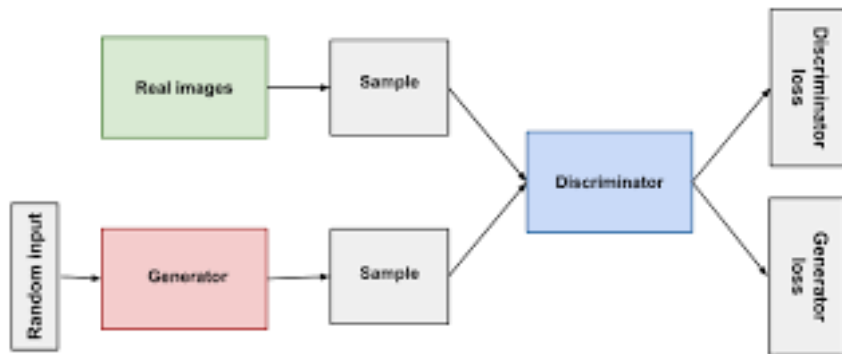
- $D(x) = \frac{p_X(x)}{p_X(x) + p_\lambda(x)}$
- $p_X(x)$ is distribution of training data.
- $p_\lambda(x)$ is distribution learnt by model λ
- When $p_X(x) = p_\lambda(x)$, then $D(x) = 1/2$
- Try to estimate $D(x)$ using a suitable cost function.

Training of GANs

- Two player game.
- Generative vs discriminative network.
- **Generative Network:** Generates sample that looks like from a training data distribution.
- **Discriminative Network:** Try to discriminate between original samples and generated samples.
- Generative network generates synthetic samples and presents it to discriminative network saying it is from original data !
- Discriminative network declares whether given sample is fake or real.
- When Generative network is successful in fooling discriminative network, then we have a good generative network !



GAN Block Diagram



Objective Function for GAN network

- For generative network, the objective function should be to minimize the parameter θ_g of the network.
- For discriminative network, the objective function should be to maximize the parameter θ_d of the network.
- Let $\log(D_{\theta_d}(x))$ represent output of discriminator network for real data x .
- Maximize discriminator output for real data $\max_{\theta_d} E_x(\log(D_{\theta_d}(x)))$.
- Minimize discriminator output for fake or synthetic data coming through generator network $\min_{\theta_d} E_z(\log(D_{\theta_d}(G_{\theta_g}(z))))$.
- Equivalently Maximize discriminator output for fake or synthetic data coming through generator network for $\max_{\theta_d} E_z(1 - \log(D_{\theta_d}(G_{\theta_g}(z))))$.



Objective Function for GAN network

- For generative network, the objective function should be to minimize the parameter θ_g of the network.
- Let $\log(G_{\theta_g}(z))$ represent output of generator network for random sample data z .
- Minimize generator network output for random sample data such that it matches closely with training data x , $\min_{\theta_g} E_z(\log(G_{\theta_g}(z)))$.
- Minimize discriminator output for fake or synthetic data coming through generator network $\min_{\theta_d, \theta_g} E_x(\log(D_{\theta_d}(G_{\theta_g}(z))))$.
- Joint min-max optimization is
$$\min_{\theta_g} \max_{\theta_d} [E_x(\log(D_{\theta_d}(x))) + E_z(1 - \log(D_{\theta_d}(G_{\theta_g}(z))))].$$

Objective Function for GAN network

- Discriminator parameter θ_d wants to maximize such that $D_{\theta_d}(x)$ is close to 1 for training data and $D_{\theta_d}(G_{\theta_g}(z))$ is close to 0 for synthetic data.
- Generator parameter θ_g wants to minimize such that $D_{\theta_d}(G_{\theta_g}(z))$ is close to 1 for synthetic data.
- Accordingly tries to fool discriminator to think synthetic data as real data.

Training GAN

- Joint min-max optimization is

$$\min_{\theta_g} \max_{\theta_d} [E_x(\log(D_{\theta_d}(x))) + E_z(1 - \log(D_{\theta_d}(G_{\theta_g}(z))))].$$

- Alternate between

- Gradient ascent for discriminator for

$$\max_{\theta_d} [E_x(\log(D_{\theta_d}(x))) + E_z(1 - \log(D_{\theta_d}(G_{\theta_g}(z))))].$$

- Gradient descent for $\min_{\theta_g} [E_z(1 - \log(D_{\theta_d}(G_{\theta_g}(z))))].$

- Alternatively, alternate between

- Gradient ascent for discriminator for

$$\max_{\theta_d} [E_x(\log(D_{\theta_d}(x))) + E_z(1 - \log(D_{\theta_d}(G_{\theta_g}(z))))].$$

- Gradient ascent for $\max_{\theta_g} [E_z(\log(D_{\theta_d}(G_{\theta_g}(z))))].$



Training GAN

- In given iteration, train discriminator using given train (real) data and its noisy version generated by generator to discriminate the two.
- In same iteration, train generator using given feedback about noisy version data it generated earlier to maximize looking like real data.
- Repeat this iteratively till the discriminator confuses noisy data and classify it as real data.
- After this the generator network is ready to generate new data looking like original data.

Testing of GAN

- Benchmark distribution as reference.
- Generate samples from GAN network and compare how it closely matches with benchmark distribution samples.
- Ideally should be same as sampled distribution.

Transformer as Generative Model

- Transformer is also Encoder-Decoder architecture
- Like VAE, train transformer as E-D combination
- Use only decoder like VAE to generate new data
- Generative pre-trained transformer (GPT)
- Large language model (LLM)
- ChatGPT

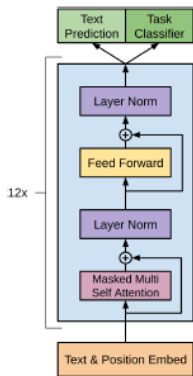
Generative Pre-Trained Transformer (GPT)

- GPTs are based on the transformer architecture
- Pre-trained on large data sets of unlabelled text
- Able to generate novel human-like content
- **Language Model:** Predicting next word in the word sequence
- GPT - Language modelling using transformer

GPT Training

- A high-capacity language model on a large corpus of text
- A fine-tuning stage, where the model is adapted to a discriminative task with labeled data
- The first one is achieved by unsupervised pre-training
- A multi-layer Transformer decoder is used for language modelling
- After pre-training the model, the parameters are adapted to the supervised target task.

Decoder only Transformer Architecture for GPT



- Applies a multi-headed self-attention operation over the input context tokens followed by position-wise feedforward layers to produce an output distribution over target tokens

Performance of GPT on NLI tasks

Table 2: Experimental results on natural language inference tasks, comparing our model with current state-of-the-art methods. 5x indicates an ensemble of 5 models. All datasets use accuracy as the evaluation metric.

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	<u>89.3</u>	-	-	-
CAFE [58] (5x)	80.2	79.0	<u>89.3</u>	-	-	-
Stochastic Answer Network [35] (3x)	<u>80.6</u>	<u>80.1</u>	-	-	-	-
CAFE [58]	78.7	77.9	88.5	<u>83.3</u>		
GenSen [64]	71.4	71.3	-	-	<u>82.3</u>	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	61.7
Finetuned Transformer LM (ours)	82.1	81.4	89.9	88.3	88.1	56.0

Thank You